

SPRING BOOT MICROSERVICES

Complete Interview Guide

with Scenario-Based Questions & Full Java Code Examples

12
Chapters

100+
Questions

80+
Code Samples

15
Patterns

All Topics
Covered

Topics Covered

- Service Discovery • API Gateway • Config Server • Circuit Breaker • Load Balancing
- Distributed Tracing • Security/JWT • Kafka Messaging • Docker • Kubernetes
- Database Per Service • Saga Pattern • CQRS • Testing • Monitoring & Observability

BEGINNER

Core concepts, annotations, basic setup

INTERMEDIATE

Service mesh, resilience, messaging

ADVANCED

CQRS, Saga, EOS, production patterns

EXPERT

K8s, Observability, System Design

Table of Contents

Chapter 1: Microservices Fundamentals

- What are microservices?
- Monolith vs Microservices
- 12-Factor App
- When NOT to use microservices

Chapter 2: Spring Boot Core & Auto-Configuration

- @SpringBootApplication
- Auto-Configuration internals
- Actuator
- Profiles & Config

Chapter 3: Service Discovery — Eureka & Kubernetes

- Eureka Server setup
- Client-side load balancing
- K8s Service Discovery
- Health checks

Chapter 4: API Gateway — Spring Cloud Gateway

- Route configuration
- JWT filter
- Rate limiting
- Circuit breaker integration

Chapter 5: Inter-Service Communication

- RestTemplate vs WebClient
- OpenFeign
- Synchronous vs Async
- Resilience patterns

Chapter 6: Resilience — Circuit Breaker, Retry, Bulkhead

- Resilience4j
- Circuit breaker states
- Retry with backoff
- Bulkhead pattern

Chapter 7: Security — JWT, OAuth2 & Spring Security

- JWT authentication
- OAuth2 with Keycloak
- Method security
- Propagating tokens

Chapter 8: Messaging — Kafka & RabbitMQ

- Producer/Consumer
- Error handling DLQ
- Transactional messages
- Event-driven patterns

Chapter 9: Distributed Tracing & Observability

- Micrometer Tracing
- Zipkin/Tempo
- Prometheus + Grafana
- Structured logging

Chapter 10: Data Patterns — DB per Service, SAGA, CQRS

- Database per service
- Saga choreography
- CQRS + Event Sourcing
- Outbox pattern

Chapter 11: Docker, Kubernetes & Deployment

- Dockerfile best practices
- K8s Deployment/Service
- ConfigMap & Secrets
- Health probes

Chapter 12: Testing Microservices

- Unit, Integration, Contract tests
- WireMock
- Testcontainers
- Consumer-driven contracts

Microservices Fundamentals

Core concepts, architecture principles & design decisions

Q1 of 100

What are microservices? How do they differ from a monolithic architecture?

Microservices is an architectural style where an application is built as a collection of small, independently deployable services, each running in its own process and communicating via APIs. Each service owns its domain data and can be deployed, scaled, and updated independently.

Aspect	Monolith vs Microservices
Deployment	Single deployable unit vs independent per service
Scaling	Scale whole app vs scale individual services
Tech Stack	Single stack vs polyglot (best tool per service)
Team Structure	Large shared team vs small autonomous teams (Conway's Law)
Failure Impact	One bug can take down everything vs isolated failures
Data Management	Single shared DB vs DB per service
Complexity	Simple dev setup vs distributed system complexity

■ **Interview Tip:** Lead with the trade-offs. Microservices solve scalability and team autonomy but add distributed systems complexity. Show you understand both sides.

What are the 12-Factor App principles and why do they matter for microservices?

The 12-Factor App methodology defines principles for building modern, cloud-native, scalable services. They are especially critical for microservices deployed in containers and orchestrated by Kubernetes.

```
# Key 12-Factor principles for microservices:

# I.   Codebase    – one codebase per service, tracked in version control
# II.  Dependencies – explicitly declare, isolate (pom.xml / build.gradle)
# III. Config      – store config in environment (NEVER hardcode)
# IV.  Backing Services – treat DB, Kafka, Redis as attached resources
# V.    Build/Release/Run – strict separation of stages
# VI.  Processes   – stateless, share-nothing processes
# VII. Port Binding – self-contained, expose service via port
# VIII. Concurrency – scale out via processes, not threads
# IX.  Disposability – fast startup, graceful shutdown
# X.   Dev/Prod Parity – keep environments as similar as possible
# XI.  Logs        – treat as event streams (stdout -> ELK/Loki)
# XII. Admin Tasks – run as one-off processes

# application.yml – Factor III (Config from environment)
spring:
  datasource:
    url: ${DB_URL}           # from env / K8s ConfigMap
    username: ${DB_USER}
    password: ${DB_PASSWORD} # from K8s Secret
  kafka:
    bootstrap-servers: ${KAFKA_BROKERS:localhost:9092}
```

■ **Interview Tip:** Interviewers love when you connect 12-Factor to concrete Spring Boot practices: `@ConfigurationProperties`, Spring Cloud Config, `actuator /health`, graceful shutdown.

Q3 of 100

SCENARIO: Design a microservices architecture for an e-commerce platform. What services would you create?

■ Scenario: E-Commerce Platform — Service Decomposition

Apply the **Single Responsibility Principle** at the service level. Decompose by business capability (Domain-Driven Design bounded contexts).

```
# E-Commerce Service Map

API Gateway (port 8080) – routing, auth, rate limiting
|
+-- User Service      (8081) – registration, auth, profiles
+-- Product Service   (8082) – catalog, search, inventory
+-- Order Service     (8083) – order lifecycle, state machine
+-- Payment Service   (8084) – payments, refunds, billing
+-- Notification Svc  (8085) – email/SMS/push via Kafka events
+-- Shipping Service  (8086) – fulfillment, tracking
+-- Review Service    (8087) – ratings, reviews

# Supporting Services
Discovery Server (Eureka / K8s DNS) – service registry
Config Server    – centralized config
Kafka            – async event bus
Redis            – distributed cache, sessions
Zipkin / Tempo   – distributed tracing
Prometheus/Grafana – metrics & alerting
```

■ **Interview Tip:** In system design interviews: name your services, explain why you drew the boundary where you did (bounded context), and mention what events flow between them.

Q4 of 100

When should you NOT use microservices? What are the main pitfalls?

Microservices are not always the answer. Distributed systems introduce complexity that can outweigh the benefits for small teams or simple domains.

- **Small teams:** Overhead of 10 services with a 3-person team is counterproductive
- **Simple domain:** An e-commerce site with 5 endpoints doesn't need 8 services
- **Early stage startup:** Move fast with a modular monolith, extract services when pain points emerge
- **Latency-critical paths:** Network hops between services add 1-10ms each
- **Strong transactional consistency required:** Distributed transactions are complex
- **Team lacks distributed systems expertise:** Debugging is exponentially harder

■ *Martin Fowler: 'Don't start with microservices. Start with a monolith, keep it modular, and extract services when team size or scalability demands it.'*

Spring Boot Core & Auto-Configuration

*Annotations, auto-config internals,
Actuator & Profiles*

Q5 of 100

Explain @SpringBootApplication and how Spring Boot auto-configuration works internally.

```
// @SpringBootApplication = 3 annotations combined
@SpringBootApplication
// is equivalent to:
@Configuration           // marks as config class (bean definitions)
@EnableAutoConfiguration // triggers auto-config
@ComponentScan           // scans current package + sub-packages
public class OrderServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(OrderServiceApplication.class, args);
    }
}

# How Auto-Configuration works internally:
# 1. @EnableAutoConfiguration triggers AutoConfigurationImportSelector
# 2. Reads META-INF/spring/org.springframework.boot.autoconfigure
#    .AutoConfiguration.imports (Spring Boot 3.x)
# 3. Each auto-config class has @ConditionalOn* guards:

# Example: DataSourceAutoConfiguration
@AutoConfiguration
@ConditionalOnClass(DataSource.class)           # only if DataSource is on classpath
@ConditionalOnMissingBean(DataSource.class)     # only if user didn't define one
@EnableConfigurationProperties(DataSourceProperties.class)
public class DataSourceAutoConfiguration {
    @Bean
    @ConditionalOnProperty(name='spring.datasource.url')
    public DataSource dataSource(DataSourceProperties props) {
        return props.initializeDataSourceBuilder().build();
    }
}
```

■ **Interview Tip:** @ConditionalOnClass, @ConditionalOnMissingBean, @ConditionalOnProperty are the three most common conditions. Know them cold.

How do you configure Spring Boot Actuator for production microservices health checks?

```
# application.yml – Actuator production config
management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus,loggers
      base-path: /actuator
  endpoint:
    health:
      show-details: when_authorized # hide internals from public
      show-components: always
      probes:
        enabled: true # /actuator/health/liveness + /readiness
  health:
    livenessState:
      enabled: true
    readinessState:
      enabled: true
    db:
      enabled: true
    kafka:
      enabled: true
  metrics:
    distribution:
      percentiles-histogram:
        http.server.requests: true
    tags:
      application: ${spring.application.name}
      environment: ${APP_ENV:local}
// Custom Health Indicator
@Component
public class ExternalApiHealthIndicator implements HealthIndicator {

    private final ExternalApiClient client;

    @Override
    public Health health() {
        try {
            client.ping();
            return Health.up()
                .withDetail('externalApi', 'Reachable')
                .withDetail('responseTime', '12ms')
        } catch (Exception e) {
            return Health.down(e)
                .withDetail('externalApi', 'Unreachable')
                .withDetail('responseTime', '12ms')
        }
    }
}
```



```
        .build();
    } catch (Exception e) {
        return Health.down()
            .withDetail('error', e.getMessage())
            .build();
    }
}
```

■ **Interview Tip:** Kubernetes uses /actuator/health/liveness (is app alive?) and /actuator/health/readiness (can it serve traffic?) as separate probes.

How do you implement Spring Cloud Config Server for centralized configuration?

```
<!-- Config Server pom.xml -->
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-config-server</artifactId>
</dependency>
// Config Server Application
@SpringBootApplication
@EnableConfigServer
public class ConfigServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(ConfigServerApplication.class, args);
    }
}
# Config Server – application.yml
server:
  port: 8888
spring:
  application:
    name: config-server
  cloud:
    config:
      server:
        git:
          uri: https://github.com/myorg/config-repo
          default-label: main
          search-paths: '{application}' # per-service folder
          clone-on-start: true
          timeout: 10
# Client Service – bootstrap.yml
spring:
  application:
    name: order-service # must match config file name
  config:
    import: 'configserver:http://config-server:8888'
  profiles:
    active: ${APP_PROFILE:dev} # loads order-service-dev.yml
# Config repo structure:
# config-repo/
#   order-service/
#     order-service.yml # base config
#     order-service-dev.yml # dev overrides
```

```
#      order-service-prod.yml      # prod overrides
```

■ **Interview Tip:** In production, secure the Config Server with basic auth or OAuth2, and use Spring Cloud Bus + Kafka to push config refresh events without restart.

Explain Spring Boot profiles and how to manage environment-specific configuration.

```
# application.yml – multi-profile config
spring:
  profiles:
    active: ${APP_PROFILE:dev}
---
spring:
  config:
    activate:
      on-profile: dev
  datasource:
    url: jdbc:h2:mem:devdb
    driver-class-name: org.h2.Driver
  logging:
    level:
      com.myapp: DEBUG
---
spring:
  config:
    activate:
      on-profile: prod
  datasource:
    url: ${DB_URL}
    username: ${DB_USER}
    password: ${DB_PASS}
  jpa:
    show-sql: false
  logging:
    level:
      root: WARN
// @Profile on beans
@Configuration
public class CacheConfig {

    @Bean
    @Profile('dev')                // H2 in-memory cache for dev
    public CacheManager devCache() {
        return new ConcurrentMapCacheManager('orders','products');
    }

    @Bean
    @Profile('prod')              // Redis for production
    public CacheManager redisCacheManager(RedisConnectionFactory f) {
```

```
        return RedisCacheManager.builder(f)
            .cacheDefaults(RedisCacheConfiguration.defaultCacheConfig()
                .entryTtl(Duration.ofMinutes(30)))
            .build();
    }
}
```

Service Discovery — Eureka & Kubernetes

*Registry, client-side LB, health
checks & K8s DNS*

Set up a complete Eureka Server and register multiple microservices with it.

```

<!-- Eureka Server pom.xml -->
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
</dependency>
@SpringBootApplication
@EnableEurekaServer
public class DiscoveryServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(DiscoveryServerApplication.class, args);
    }
}

# Eureka Server – application.yml
server:
  port: 8761
spring:
  application:
    name: discovery-server
  security:
    user:
      name: ${EUREKA_USER:eureka}
      password: ${EUREKA_PASS:secret}
eureka:
  instance:
    hostname: localhost
  client:
    register-with-eureka: false # server doesn't register itself
    fetch-registry: false
  server:
    enable-self-preservation: false # dev only – disable in prod
    eviction-interval-timer-in-ms: 5000

# Eureka Client – order-service/application.yml
spring:
  application:
    name: order-service # this is the service name in registry
eureka:
  client:
    service-url:
      defaultZone: http://${EUREKA_USER}:${EUREKA_PASS}@eureka:8761/eureka/
    registry-fetch-interval-seconds: 15
  instance:
    prefer-ip-address: true

```

```
lease-renewal-interval-in-seconds: 10
lease-expiration-duration-in-seconds: 30
metadata-map:
  version: '@project.version@'
  zone: us-east-1a
```

■ **Interview Tip:** In Kubernetes, replace Eureka with K8s native DNS (serviceName.namespace.svc.cluster.local). Eureka is for non-K8s environments.

SCENARIO: order-service needs to call product-service using load-balanced discovery. Implement with `@LoadBalanced`.

■ Scenario: Client-Side Load Balancing with Spring Cloud LoadBalancer

```
@Configuration
public class WebClientConfig {

    // @LoadBalanced enables client-side load balancing
    // Resolves 'product-service' via Eureka / K8s DNS

    @Bean
    @LoadBalanced

    public WebClient.Builder loadBalancedWebClientBuilder() {
        return WebClient.builder()
            .defaultHeader(HttpHeaders.CONTENT_TYPE,
                MediaType.APPLICATION_JSON_VALUE)
            .codecs(c -> c.defaultCodecs()
                .maxInMemorySize(1024 * 1024));
    }
}

@Service
@RequiredArgsConstructor
public class OrderService {

    private final WebClient.Builder webClientBuilder;

    public ProductDto getProduct(String productId) {
        return webClientBuilder.build()
            .get()
            .uri('http://product-service/api/v1/products/{id}', productId)
            .retrieve()
            .onStatus(HttpStatusCode::is4xxClientError,
                res -> Mono.error(new ProductNotFoundException(productId)))
            .onStatus(HttpStatusCode::is5xxServerError,
                res -> Mono.error(new ServiceUnavailableException('product-service')))
            .bodyToMono(ProductDto.class)
            .timeout(Duration.ofSeconds(5))
            .block();
    }
}
```

■ **Common Mistake:** Never hardcode service URLs like 'http://localhost:8082'. Always use the service name registered in the discovery server.

API Gateway — Spring Cloud Gateway

Routing, filters, JWT auth, rate limiting & circuit breaking

Configure Spring Cloud Gateway with routes, circuit breakers, and rate limiting.

```
# gateway/application.yml – complete production config
spring:
  application:
    name: api-gateway
  cloud:
    gateway:
      discovery:
        locator:
          enabled: true
          lower-case-service-id: true
    globalcors:
      corsConfigurations:
        '[/**]':
          allowedOrigins: ${ALLOWED_ORIGINS:http://localhost:3000}
          allowedMethods: [GET,POST,PUT,DELETE,PATCH,OPTIONS]
          allowedHeaders: '*'
          allowCredentials: true
    default-filters:
      - AddResponseHeader=X-Gateway, MyApp-GW/1.0
      - RemoveResponseHeader=Server, X-Powered-By
    routes:
      - id: order-service
        uri: lb://order-service
        predicates:
          - Path=/api/v1/orders/**
        filters:
          - name: CircuitBreaker
            args:
              name: order-cb
              fallbackUri: forward:/fallback/orders
          - name: RequestRateLimiter
            args:
              redis-rate-limiter.replenishRate: 100
              redis-rate-limiter.burstCapacity: 200
              redis-rate-limiter.requestedTokens: 1
          - name: Retry
            args:
              retries: 3
              statuses: BAD_GATEWAY, SERVICE_UNAVAILABLE
              methods: GET
              backoff:
```

```
        firstBackoff: 50ms
        maxBackoff: 500ms
        factor: 2
- id: product-service
  uri: lb://product-service
  predicates:
    - Path=/api/v1/products/**
    - Header=X-Version, v2
  filters:
    - name: CircuitBreaker
      args: {name: product-cb, fallbackUri: forward:/fallback/products}
- id: auth-public
  uri: lb://user-service
  predicates:
    - Path=/api/v1/auth/**
# No JWT filter for auth endpoints
```

■ **Interview Tip:** Rate limiting uses Redis token bucket algorithm. ReplenishRate = steady-state RPS. BurstCapacity = max burst. Set these per route based on SLA.

Implement a JWT authentication filter in Spring Cloud Gateway.

```
@Component
@Slf4j
@RequiredArgsConstructor
public class JwtAuthFilter implements GlobalFilter, Ordered {

    private final JwtService jwtService;
    private final AntPathMatcher pathMatcher = new AntPathMatcher();

    private static final List<String> PUBLIC_PATHS = List.of(
        '/api/v1/auth/**',
        '/api/v1/products/public/**',
        '/actuator/**',
        '/v3/api-docs/**'
    );

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        String path = exchange.getRequest().getURI().getPath();

        // Skip JWT check for public paths
        if (PUBLIC_PATHS.stream().anyMatch(p -> pathMatcher.match(p, path))) {
            return chain.filter(exchange);
        }

        String authHeader = exchange.getRequest()
            .getHeaders().getFirst(HttpHeaders.AUTHORIZATION);

        if (authHeader == null || !authHeader.startsWith('Bearer ')) {
            return unauthorized(exchange, 'Missing or invalid Authorization header');
        }

        String token = authHeader.substring(7);
        if (!jwtService.isValid(token)) {
            return unauthorized(exchange, 'Invalid or expired JWT token');
        }

        // Propagate user info to downstream services via headers
        String userId = jwtService.extractUserId(token);
        String username = jwtService.extractUsername(token);
        String roles = jwtService.extractRoles(token);

        ServerHttpRequest mutated = exchange.getRequest().mutate()
            .header('X-User-Id', userId)
            .header('X-Username', username)
            .header('X-User-Roles', roles)
            .build();

        return chain.filter(exchange.mutate().request(mutated).build());
    }
}
```

```
private Mono<Void> unauthorized(ServerWebExchange ex, String message) {  
    ex.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);  
    ex.getResponse().getHeaders().add(  
        HttpHeaders.CONTENT_TYPE, MediaType.APPLICATION_JSON_VALUE);  
    String body = '{"error":"' + message + '", "status":401}';  
    DataBuffer buffer = ex.getResponse().bufferFactory()  
        .wrap(body.getBytes(StandardCharsets.UTF_8));  
    return ex.getResponse().writeWith(Mono.just(buffer));  
}  
  
@Override  
public int getOrder() { return -100; } // run before other filters  
}
```

■ **Interview Tip:** Downstream services should read X-User-Id / X-Username headers (injected by gateway) — they should NEVER re-validate the JWT themselves, saving CPU.

Implement a fallback controller for circuit-broken services in the gateway.

```

@RestController
@Slf4j
public class FallbackController {

    @RequestMapping('/fallback/orders')
    public ResponseEntity<ApiResponse<Void>> orderFallback(
        ServerWebExchange exchange) {
        log.warn('Circuit breaker triggered for order-service: {}',
            exchange.getRequest().getURI());
        return ResponseEntity
            .status(HttpStatus.SERVICE_UNAVAILABLE)
            .body(ApiResponse.<Void>builder()
                .success(false)
                .message('Order service is temporarily unavailable. Please try again in a mo
ment.')
                .errorCode('ORDER_SERVICE_DOWN')
                .retryAfter(30)
                .build());
    }

    @RequestMapping('/fallback/products')
    public ResponseEntity<ApiResponse<List<ProductDto>>> productFallback() {
        // Return cached/static data for read endpoints
        return ResponseEntity.ok(
            ApiResponse.<List<ProductDto>>builder()
                .success(true)
                .message('Showing cached products')
                .data(productCacheService.getCachedFeatured())
                .build());
    }
}

```

■ *Fallback responses should be meaningful — for reads, return cached data; for writes, return a clear error with a retry hint and a correlation ID.*

Inter-Service Communication

*OpenFeign, WebClient,
RestTemplate & async patterns*

Compare RestTemplate, WebClient, and OpenFeign. When do you use each?

Feature	RestTemplate vs WebClient vs OpenFeign
Type	Sync blocking Async reactive (non-blocking) Declarative (sync/async)
Spring Status	Deprecated in 6.x Preferred Preferred for service-to-service
Best For	Legacy code High concurrency reactive apps Clean service clients
Resilience	Manual Manual Built-in via Resilience4j decorator
Load Balancing	@LoadBalanced @LoadBalanced Automatic via discovery
Code Style	Imperative Fluent reactive Interface with annotations

```
// OpenFeign Client – recommended for service-to-service calls
@FeignClient(
    name = 'product-service',
    fallbackFactory = ProductClientFallbackFactory.class
)
public interface ProductClient {

    @GetMapping('/api/v1/products/{id}')
    ProductDto getProduct(@PathVariable String id);

    @GetMapping('/api/v1/products')
    PageResponse<ProductDto> listProducts(
        @RequestParam int page,
        @RequestParam int size,
        @RequestParam(required=false) String category);

    @PostMapping('/api/v1/products/reserve')
    ReservationResponse reserveStock(@RequestBody StockReservationRequest req);
}

// Feign configuration with timeout and logging
@Configuration
public class FeignConfig {

    @Bean
    public Request.Options options() {
        return new Request.Options(
            Duration.ofSeconds(2),    // connect timeout
            Duration.ofSeconds(5),    // read timeout
            true);
    }
}
```

```

    }

    @Bean
    public Logger.Level feignLogLevel() {
        return Logger.Level.BASIC; // log request/response in non-prod
    }

    // Propagate JWT token to downstream service
    @Bean
    public RequestInterceptor authInterceptor() {
        return template -> {
            String token = SecurityContextHolder.getContext()
                .getAuthentication().getCredentials().toString();
            template.header('Authorization', 'Bearer ' + token);
        };
    }
}

// Fallback Factory for graceful degradation
@Component
public class ProductClientFallbackFactory
    implements FallbackFactory<ProductClient> {

    @Override
    public ProductClient create(Throwable cause) {
        return new ProductClient() {

            @Override
            public ProductDto getProduct(String id) {
                log.error('ProductClient fallback: {}', cause.getMessage());
                return ProductDto.empty(id); // cached/default response
            }

            @Override
            public PageResponse<ProductDto> listProducts(int p, int s, String cat) {
                return PageResponse.empty();
            }
        };
    }
}

```

■ **Interview Tip:** Use FallbackFactory (not Fallback) when you need access to the exception to log or classify the failure type.

SCENARIO: Order service must call inventory, payment, and notification services. Implement parallel async calls with WebClient.**■ Scenario: Parallel Non-Blocking Service Calls**

```
@Service
@RequiredArgsConstructor
public class OrderOrchestrationService {

    private final WebClient.Builder webClient;

    public OrderSummary processOrder(CreateOrderRequest req) {
        // Call inventory and customer profile IN PARALLEL (non-blocking)
        Mono<InventoryResult> inventoryCheck = webClient.build()
            .post()
            .uri('http://inventory-service/api/v1/inventory/check')
            .bodyValue(new StockCheckRequest(req.getItems()))
            .retrieve()
            .bodyToMono(InventoryResult.class)
            .timeout(Duration.ofSeconds(3));

        Mono<CustomerProfile> customerProfile = webClient.build()
            .get()
            .uri('http://user-service/api/v1/users/{id}', req.getCustomerId())
            .retrieve()
            .bodyToMono(CustomerProfile.class)
            .timeout(Duration.ofSeconds(2));

        // Zip combines when BOTH complete
        return Mono.zip(inventoryCheck, customerProfile)
            .flatMap(tuple -> {
                InventoryResult inv = tuple.getT1();
                CustomerProfile cust = tuple.getT2();

                if (!inv.isAvailable()) {
                    return Mono.error(new OutOfStockException(inv.getUnavailableItems()));
                }

                // Create order with enriched data
                return createOrderMono(req, cust, inv);
            })
            .block();
    }
}
```

■ **Interview Tip:** Parallel WebClient calls with Mono.zip() save the sum of sequential latencies. 3 calls at 200ms each = 200ms total instead of 600ms.

How do you propagate correlation IDs and trace context across microservices?

```
// MDC Filter – sets correlation ID on every request
@Component
@Order(Ordered.HIGHEST_PRECEDENCE)
public class CorrelationIdFilter extends OncePerRequestFilter {

    public static final String CORRELATION_HEADER = 'X-Correlation-Id';
    public static final String MDC_KEY = 'correlationId';

    @Override
    protected void doFilterInternal(HttpServletRequest req,
                                    HttpServletResponse res, FilterChain chain)
        throws ServletException, IOException {
        String correlationId = req.getHeader(CORRELATION_HEADER);
        if (correlationId == null) {
            correlationId = UUID.randomUUID().toString().substring(0, 8);
        }
        MDC.put(MDC_KEY, correlationId);
        res.addHeader(CORRELATION_HEADER, correlationId);
        try {
            chain.doFilter(req, res);
        } finally {
            MDC.remove(MDC_KEY); // prevent memory leak in thread pools
        }
    }
}

# logback-spring.xml – include correlationId in all logs
<pattern>%d{HH:mm:ss} [%thread] [%X{correlationId}] [%X{traceId}] %-5level %logger - %msg%n<
/pattern>

# application.yml – Micrometer Tracing (auto-propagates to all HTTP clients)
management:
  tracing:
    sampling:
      probability: 1.0 # 100% in dev, 10% in prod
    zipkin:
      tracing:
        endpoint: http://zipkin:9411/api/v2/spans
logging:
  pattern:
    level: '%5p [%${spring.application.name},%X{traceId},%X{spanId}]'
```

■ With Micrometer Tracing (Spring Boot 3.x), `traceId/spanId` are automatically injected into logs and propagated via HTTP headers (B3/W3C format) to all downstream calls.

Resilience — Circuit Breaker, Retry & Bulkhead

*Resilience4j patterns for
fault-tolerant microservices*

Explain the Circuit Breaker pattern and implement it with Resilience4j.

A Circuit Breaker wraps a remote call and monitors failures. When failures exceed a threshold, it 'opens' and returns a fallback immediately — preventing cascading failures and giving the failing service time to recover.

State	Description
CLOSED	Normal operation — calls pass through, failures tracked
OPEN	Failures exceeded threshold — calls short-circuit to fallback
HALF_OPEN	After wait duration — lets limited calls through to test recovery

```
# application.yml — Resilience4j config
resilience4j:
  circuitbreaker:
    instances:
      product-service:
        sliding-window-type: COUNT_BASED
        sliding-window-size: 10          # last 10 calls
        failure-rate-threshold: 50       # open if 50%+ fail
        wait-duration-in-open-state: 30s
        permitted-number-of-calls-in-half-open-state: 3
        minimum-number-of-calls: 5      # need at least 5 before calculating
        slow-call-rate-threshold: 80    # also count slow calls as failures
        slow-call-duration-threshold: 3s
        record-exceptions:
          - java.io.IOException
          - java.util.concurrent.TimeoutException
        ignore-exceptions:
          - com.example.exceptions.BusinessException # don't trip CB on biz errors

@Service
@RequiredArgsConstructor
public class ProductService {

    private final ProductClient productClient;

    @CircuitBreaker(name = 'product-service', fallbackMethod = 'getProductFallback')
    @Retry(name = 'product-service', fallbackMethod = 'getProductFallback')
    @TimeLimiter(name = 'product-service')
    public ProductDto getProduct(String productId) {
        return productClient.getProduct(productId);
    }

    // Fallback must have same return type + Throwable parameter
```

```
private ProductDto getProductFallback(String productId, Throwable ex) {  
    log.warn('CB fallback for product {}: {}'.format(productId, ex.getMessage()));  
    return productCacheService.getCached(productId)  
        .orElseThrow(() -> new ServiceUnavailableException('product-service'));  
}
```

■ **Interview Tip:** Layer Retry INSIDE CircuitBreaker: @CircuitBreaker(outer) + @Retry(inner). Retry handles transient issues; CB prevents overloading a failing service.

Implement Bulkhead pattern to isolate thread pools between services.

Bulkhead isolates resources so a slow/failing service cannot starve thread resources from healthy services. Two implementations: Thread-pool (separate executors) and Semaphore (concurrent call limit).

```
# Bulkhead config
resilience4j:
  bulkhead:
    instances:
      inventory-service:
        max-concurrent-calls: 20    # max 20 concurrent calls
        max-wait-duration: 100ms    # queue wait before rejection
  thread-pool-bulkhead:
    instances:
      payment-service:
        max-thread-pool-size: 10
        core-thread-pool-size: 5
        queue-capacity: 100
        keep-alive-duration: 20ms

@Service
public class InventoryService {

    @Bulkhead(name = 'inventory-service', type = Bulkhead.Type.SEMAPHORE,
              fallbackMethod = 'checkStockFallback')
    public StockResult checkStock(String productId, int quantity) {
        return inventoryClient.check(productId, quantity);
    }

    // ThreadPool bulkhead returns CompletableFuture
    @Bulkhead(name = 'payment-service', type = Bulkhead.Type.THREADPOOL)
    @CircuitBreaker(name = 'payment-service')
    public CompletableFuture<PaymentResult> processPayment(PaymentRequest req) {
        return CompletableFuture.supplyAsync(() ->
            paymentClient.process(req));
    }

    private StockResult checkStockFallback(String id, int qty, BulkheadFullException e) {
        log.warn('Bulkhead full for inventory-service');
        return StockResult.builder().available(false)
            .reason('Service busy, please retry').build();
    }
}
```

■ *Real-world: Payment service gets a dedicated thread pool so a slow payment gateway never blocks order reads or product searches.*

How do you implement retry with exponential backoff in Resilience4j?

```
resilience4j:
  retry:
    instances:
      order-service:
        max-attempts: 4
        wait-duration: 500ms
        enable-exponential-backoff: true
        exponential-backoff-multiplier: 2
        exponential-max-wait-duration: 10s
        retry-exceptions:
          - java.io.IOException
          - org.springframework.web.client.ResourceAccessException
        ignore-exceptions:
          - com.example.exception.InvalidOrderException

@Service
public class OrderRetryService {

    @Retry(name = 'order-service', fallbackMethod = 'submitFallback')
    public OrderResult submitOrder(OrderRequest request) {
        log.info('Attempting order submission...');
        return orderClient.submit(request);
    }

    // Retry event listener for monitoring
    @EventListener
    public void onRetryEvent(RetryOnRetryEvent event) {
        log.warn('Retry attempt {} for {}: {}'.format(
            event.getNumberOfRetryAttempts(),
            event.getName(),
            event.getLastThrowable().getMessage()));
        meterRegistry.counter('retry.attempts',
            'service', event.getName()).increment();
    }

    private OrderResult submitFallback(OrderRequest req, Exception ex) {
        // All retries exhausted
        log.error('All retries failed: {}'.format(ex.getMessage()));
        return OrderResult.pending(req.getOrderid()); // async processing fallback
    }
}
```

■ **Common Mistake:** Set retry only on idempotent operations (GET, PUT with idempotency key). Never blindly retry POST without idempotency keys — you'll create duplicate orders.

Security — JWT, OAuth2 & Spring Security

Authentication, authorization & token propagation

Implement a complete JWT authentication system in a microservice.

```
// JwtService – token creation & validation
@Service
public class JwtService {

    @Value('${jwt.secret}') private String secret;
    @Value('${jwt.access-expiry:3600000}') private long accessExpiry;
    @Value('${jwt.refresh-expiry:604800000}') private long refreshExpiry;

    private SecretKey key() {
        return Keys.hmacShaKeyFor(Base64.getDecoder().decode(secret));
    }

    public String generateAccessToken(UserDetails user) {
        return Jwts.builder()
            .subject(user.getUsername())
            .claims(Map.of(
                'roles', user.getAuthorities().stream()
                    .map(GrantedAuthority::getAuthority).toList(),
                'userId', ((AppUser) user).getId(),
                'type', 'ACCESS'))
            .issuedAt(new Date())
            .expiration(new Date(System.currentTimeMillis() + accessExpiry))
            .signWith(key())
            .compact();
    }

    public String generateRefreshToken(UserDetails user) {
        return Jwts.builder()
            .subject(user.getUsername())
            .claim('type', 'REFRESH')
            .issuedAt(new Date())
            .expiration(new Date(System.currentTimeMillis() + refreshExpiry))
            .signWith(key())
            .compact();
    }

    public Claims extractAll(String token) {
        return Jwts.parser().verifyWith(key()).build()
            .parseSignedClaims(token).getPayload();
    }

    public boolean isValid(String token, UserDetails user) {
        try {
            Claims c = extractAll(token);
            return c.getSubject().equals(user.getUsername())
                && !c.getExpiration().before(new Date())
                && 'ACCESS'.equals(c.get('type'));
        }
    }
}
```

```

        } catch (JwtException e) {
            return false;
        }
    }
}

// Security Filter Chain
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
@RequiredArgsConstructor
public class SecurityConfig {

    private final JwtAuthFilter jwtAuthFilter;

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        return http
            .csrf(AbstractHttpConfigurer::disable)
            .sessionManagement(s -> s
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers('/api/v1/auth/**').permitAll()
                .requestMatchers(HttpMethod.GET, '/api/v1/products/**').permitAll()
                .requestMatchers('/actuator/health').permitAll()
                .requestMatchers('/api/v1/admin/**').hasRole('ADMIN')
                .anyRequest().authenticated())
            .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class)
            .exceptionHandling(ex -> ex
                .authenticationEntryPoint(customEntryPoint())
                .accessDeniedHandler(customAccessDeniedHandler()))
            .build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder(12);
    }
}

```

■ **Interview Tip:** Use `@PreAuthorize("hasRole('ADMIN')"` or `#userId == authentication.principal.id"` for method-level security combining role and ownership checks.

SCENARIO: Configure OAuth2 with Keycloak as the identity provider across all microservices.

■ Scenario: OAuth2 + Keycloak Resource Server

```

<!-- pom.xml -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
</dependency>
# Resource Server – application.yml
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: http://keycloak:8080/realms/myapp
          # Spring auto-fetches public keys from:
          # {issuer-uri}/.well-known/openid-configuration
@Configuration
@EnableWebSecurity
public class ResourceServerConfig {

  @Bean
  public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    return http
      .oauth2ResourceServer(oauth2 -> oauth2
        .jwt(jwt -> jwt.jwtAuthenticationConverter(
          keycloakJwtConverter()))
      .authorizeHttpRequests(auth -> auth
        .requestMatchers('/api/v1/public/**').permitAll()
        .requestMatchers('/api/v1/admin/**').hasRole('ADMIN')
        .anyRequest().authenticated())
      .build();
  }

  // Map Keycloak 'realm_access.roles' to Spring Security roles
  @Bean
  public JwtAuthenticationConverter keycloakJwtConverter() {
    JwtGrantedAuthoritiesConverter converter = new JwtGrantedAuthoritiesConverter();
    converter.setAuthoritiesClaimName('realm_access.roles');
    converter.setAuthorityPrefix('ROLE_');
    JwtAuthenticationConverter jwtConverter = new JwtAuthenticationConverter();
    jwtConverter.setJwtGrantedAuthoritiesConverter(converter);
    return jwtConverter;
  }
}

```

```
}
```

■ *Each microservice validates the JWT independently using Keycloak's public key — no round-trip to Keycloak per request after initial key fetch.*

Messaging — Kafka & RabbitMQ

Event-driven patterns, reliability & exactly-once semantics

SCENARIO: Design an event-driven order processing flow using Kafka.**■ Scenario: Event-Driven Order Processing**

```
// Events flow:
// OrderPlaced --> [Payment Service] --> PaymentProcessed
//                                     --> PaymentFailed
// PaymentProcessed --> [Inventory Service] --> StockReserved
// StockReserved --> [Shipping Service] --> ShipmentCreated
// (any failure triggers compensating events)
// OrderService – publishes OrderPlaced event

@Service
@Transactional
@RequiredArgsConstructor
public class OrderService {

    private final OrderRepository orderRepo;
    private final KafkaTemplate<String, Object> kafka;

    public Order placeOrder(PlaceOrderRequest req) {
        Order order = orderRepo.save(Order.builder()
            .customerId(req.getCustomerId())
            .items(req.getItems())
            .totalAmount(req.calculateTotal())
            .status(OrderStatus.PENDING)
            .build());

        kafka.send('order.placed', order.getId(),
            OrderPlacedEvent.builder()
                .orderId(order.getId())
                .customerId(order.getCustomerId())
                .items(order.getItems())
                .totalAmount(order.getTotalAmount())
                .placedAt(Instant.now())
                .build());

        return order;
    }
}

// PaymentService – consumes OrderPlaced, publishes result
@Service
@RequiredArgsConstructor
@Slf4j
public class PaymentEventHandler {

    private final KafkaTemplate<String, Object> kafka;

    @KafkaListener(topics = 'order.placed', groupId = 'payment-service')
    public void handleOrderPlaced(OrderPlacedEvent event,
```



```
        @Header(KafkaHeaders.RECEIVED_PARTITION) int partition,
        @Header(KafkaHeaders.OFFSET) long offset) {
    log.info('[partition={}, offset={}] Processing payment for order {}'.format(
        partition, offset, event.getOrderId());
    try {
        Payment payment = paymentService.charge(
            event.getCustomerId(), event.getTotalAmount());

        kafka.send('payment.processed', event.getOrderId(),
            new PaymentProcessedEvent(
                event.getOrderId(), payment.getId(), payment.getAmount()));
    } catch (PaymentDeclinedException e) {
        kafka.send('payment.failed', event.getOrderId(),
            new PaymentFailedEvent(event.getOrderId(), e.getReason()));
    }
}
```

SCENARIO: Implement the Transactional Outbox pattern to guarantee event delivery.

■ Scenario: Outbox Pattern — Zero Event Loss

The Dual Write Problem: If you update the DB and send a Kafka message in two steps, a crash between them means the event is never sent. The Outbox pattern writes the event to a DB table in the SAME transaction, then a poller publishes it.

```
// Outbox Event entity
@Entity @Table(name = 'outbox_events')
@Getter @Setter @Builder @NoArgsConstructor @AllArgsConstructor
public class OutboxEvent {
    @Id @GeneratedValue private Long id;
    private String aggregateId;
    private String aggregateType;
    private String eventType;
    private String topic;
    @Column(columnDefinition = 'TEXT')
    private String payload;
    private boolean published = false;
    private LocalDateTime createdAt = LocalDateTime.now();
    private LocalDateTime publishedAt;
    private int retryCount = 0;
}

// Order service – write order + outbox event atomically
@Service
@Transactional
public class OrderService {

    public Order placeOrder(PlaceOrderRequest req) {
        Order order = orderRepo.save(mapToOrder(req));

        // Same DB transaction – either both commit or both rollback
        outboxRepo.save(OutboxEvent.builder()
            .aggregateId(order.getId())
            .aggregateType('Order')
            .eventType('OrderPlaced')
            .topic('order.placed')
            .payload(objectMapper.writeValueAsString(
                new OrderPlacedEvent(order)))
            .build());

        return order; // Both saved or both rolled back
    }
}

// Outbox Poller – runs on schedule
@Component
```

```

@Slf4j
@RequiredArgsConstructor
public class OutboxPoller {

    @Scheduled(fixedDelay = 1000, initialDelay = 5000)
    @Transactional
    public void publishPendingEvents() {
        List<OutboxEvent> pending = outboxRepo
            .findByPublishedFalseAndRetryCountLessThan(3,
                PageRequest.of(0, 100));

        for (OutboxEvent event : pending) {
            try {
                kafkaTemplate.send(event.getTopic(),
                    event.getAggregateId(),
                    objectMapper.readTree(event.getPayload()));
                event.setPublished(true);
                event.setPublishedAt(LocalDateTime.now());
            } catch (Exception e) {
                event.setRetryCount(event.getRetryCount() + 1);
                log.error('Failed to publish outbox event {}: {}'.format(
                    event.getId(), e.getMessage()));
            }
            outboxRepo.save(event);
        }
    }
}

```

■ **Interview Tip:** Debezium (CDC) is the production-grade alternative to polling — it reads PostgreSQL WAL logs and publishes to Kafka without any polling overhead.

How do you implement RabbitMQ in Spring Boot with dead letter queues and retry?

```
@Configuration
public class RabbitMQConfig {

    public static final String ORDER_QUEUE    = 'orders';
    public static final String ORDER_DLQ      = 'orders.dlq';
    public static final String ORDER_EXCHANGE = 'orders.exchange';
    public static final String DLX_EXCHANGE   = 'orders.dlx';

    @Bean
    public Queue orderQueue() {
        return QueueBuilder.durable(ORDER_QUEUE)
            // Route rejected messages to DLX
            .withArgument('x-dead-letter-exchange', DLX_EXCHANGE)
            .withArgument('x-dead-letter-routing-key', 'dlq')
            .withArgument('x-message-ttl', 30000) // 30s TTL
            .build();
    }

    @Bean
    public Queue deadLetterQueue() {
        return QueueBuilder.durable(ORDER_DLQ).build();
    }

    @Bean
    public TopicExchange orderExchange() {
        return new TopicExchange(ORDER_EXCHANGE);
    }

    @Bean
    public DirectExchange deadLetterExchange() {
        return new DirectExchange(DLX_EXCHANGE);
    }

    @Bean
    public Binding orderBinding() {
        return BindingBuilder.bind(orderQueue())
            .to(orderExchange()).with('order.#');
    }

    @Bean
    public Binding dlqBinding() {
        return BindingBuilder.bind(deadLetterQueue())
            .to(deadLetterExchange()).with('dlq');
    }
}

@Service
```

```
public class OrderMessageConsumer {  
  
    @RabbitListener(queues = RabbitMQConfig.ORDER_QUEUE)  
    public void handleOrder(OrderMessage message,  
                            Channel channel,  
                            @Header(AmqpHeaders.DELIVERY_TAG) long tag) {  
  
        try {  
            orderService.process(message);  
            channel.basicAck(tag, false); // acknowledge success  
        } catch (BusinessException e) {  
            // Permanent failure – send to DLQ (don't requeue)  
            channel.basicNack(tag, false, false);  
        } catch (Exception e) {  
            // Transient failure – requeue for retry  
            channel.basicNack(tag, false, true);  
        }  
    }  
}
```

■ *RabbitMQ DLQ is typically used with a separate monitoring service that alerts ops and provides a UI to inspect/retry dead messages.*

Distributed Tracing & Observability

Micrometer, Zipkin, Prometheus, Grafana & structured logging

Set up distributed tracing with Micrometer Tracing + Zipkin across microservices.

```

<!-- pom.xml -- each microservice -->
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-tracing-bridge-brave</artifactId>
</dependency>
<dependency>
  <groupId>io.zipkin.reporter2</groupId>
  <artifactId>zipkin-reporter-brave</artifactId>
</dependency>
# application.yml -- tracing config
management:
  tracing:
    sampling:
      probability: ${TRACE_SAMPLE_RATE:0.1} # 10% in prod, 100% in dev
  zipkin:
    tracing:
      endpoint: ${ZIPKIN_URL:http://zipkin:9411}/api/v2/spans
logging:
  pattern:
    # Auto-injects traceId and spanId into every log line
    level: '%5p [%${spring.application.name:},%X{traceId},%X{spanId}]'
// Custom span for business operations
@Service
@RequiredArgsConstructor
public class OrderService {

  private final Tracer tracer;

  public Order processOrder(OrderRequest req) {
    // Create child span for payment validation
    Span paymentSpan = tracer.nextSpan()
      .name('validate-payment')
      .tag('customerId', req.getCustomerId())
      .tag('amount', req.getAmount().toString())
      .start();

    try (Tracer.SpanInScope ws = tracer.withSpan(paymentSpan)) {
      validatePayment(req);
      paymentSpan.tag('result', 'success');
    } catch (Exception e) {
      paymentSpan.tag('error', e.getMessage())
        .error(e);
    }

    throw e;
  }
}

```

```
        } finally {  
            paymentSpan.end();  
        }  
        return createOrder(req);  
    }  
}
```

■ **Interview Tip:** One `traceId` spans ALL services for a single user request. In Zipkin/Grafana Tempo you can filter by `traceId` to see the entire call graph instantly.

Configure Prometheus metrics and a Grafana dashboard for microservices.

```
// Custom business metrics with Micrometer
@Service
@RequiredArgsConstructor
public class OrderMetricsService {

    private final MeterRegistry registry;
    private final Counter ordersPlaced;
    private final Counter ordersFailed;
    private final Timer orderProcessingTime;

    @PostConstruct
    public void initMetrics() {
        ordersPlaced = Counter.builder('orders.placed')
            .description('Total orders successfully placed')
            .tag('service', 'order-service')
            .register(registry);

        ordersFailed = Counter.builder('orders.failed')
            .description('Total failed orders')
            .register(registry);

        orderProcessingTime = Timer.builder('orders.processing.time')
            .description('Time to process an order end-to-end')
            .publishPercentiles(0.5, 0.95, 0.99) // p50, p95, p99
            .register(registry);
    }

    public Order placeOrder(OrderRequest req) {
        return orderProcessingTime.record(() -> {
            Order order = doPlaceOrder(req);
            ordersPlaced.increment();
            return order;
        });
    }
}

# Prometheus scrape config – prometheus.yml
scrape_configs:
  - job_name: 'spring-microservices'
    metrics_path: '/actuator/prometheus'
    scrape_interval: 15s
    static_configs:
      - targets:
        - 'order-service:8081'
        - 'product-service:8082'
        - 'payment-service:8083'

# Key Grafana queries (PromQL):
```

```
# Request rate:    rate(http_server_requests_seconds_count[5m])
# Error rate:      rate(http_server_requests_seconds_count{status='5xx'}[5m])
# p99 latency:     histogram_quantile(0.99, rate(http_server_requests_seconds_bucket[5m]))
# Order rate:      rate(orders_placed_total[1m])
# JVM heap:        jvm_memory_used_bytes{area='heap'}
```

■ The 'Four Golden Signals' to monitor: Latency (p50/p95/p99), Traffic (RPS), Errors (5xx rate), Saturation (CPU/memory %).

Data Patterns — SAGA, CQRS & Event Sourcing

DB per service, distributed transactions & read model separation

Explain Database-per-Service pattern and how services share data without direct DB access.

Each microservice owns its database exclusively. No other service can query it directly. Data sharing happens via APIs or events. This enforces loose coupling but requires strategies for cross-service queries.

Strategy	Description & When to Use
API Composition	Orchestrator calls multiple services and aggregates (BFF pattern)
Event Streaming	Services publish events; others maintain their own read model
Shared Read DB	Write to own DB, replicate to shared read DB for reporting
CQRS + Read Model	Separate write (command) and read (query) models per service
GraphQL Federation	Federated schema spans services transparently

```
// API Composition – OrderQueryService aggregates from 3 services
@Service
@RequiredArgsConstructor
public class OrderQueryService {

    public OrderDetailView getOrderDetail(String orderId) {
        // Call three services (preferably in parallel)
        var orderMono = orderClient.getOrder(orderId);
        var productMono = productClient.getProducts(
            orderMono.block().getProductIds()); // simplify for demo
        var paymentMono = paymentClient.getPayment(orderId);

        return OrderDetailView.builder()
            .order(orderMono.block())
            .products(productMono.block())
            .payment(paymentMono.block())
            .build();
    }
}
```

■ **Interview Tip:** Never let Service A directly query Service B's database — this is the #1 microservices anti-pattern. It creates invisible coupling that breaks independently.

SCENARIO: Implement the SAGA pattern (choreography) for order → payment → inventory.

■ Scenario: Saga Choreography — Distributed Transaction Without 2PC

```
// Order Service – Step 1: Create order, publish OrderCreated
@Service
@Transactional
public class OrderSagaService {

    public Order createOrder(CreateOrderRequest req) {
        Order order = orderRepo.save(Order.builder()
            .status(PENDING).items(req.getItems()).build());

        kafka.send('order.created', order.getId(),
            new OrderCreatedEvent(order));
        return order;
    }

    // Compensation: rollback on payment failure
    @KafkaListener(topics = 'payment.failed')
    public void onPaymentFailed(PaymentFailedEvent event) {
        log.info('Compensating: cancelling order {}'.format(event.getOrderId()));
        orderRepo.updateStatus(event.getOrderId(), CANCELLED);
        kafka.send('order.cancelled', event.getOrderId(),
            new OrderCancelledEvent(event.getOrderId(), event.getReason()));
    }
}

// Payment Service – Step 2: Charge, publish success/failure
@Service
public class PaymentSagaStep {

    @KafkaListener(topics = 'order.created')
    public void onOrderCreated(OrderCreatedEvent event) {
        try {
            Payment p = paymentService.charge(event.getCustomerId(),
                event.getTotalAmount());

            kafka.send('payment.processed', event.getOrderId(),
                new PaymentProcessedEvent(event.getOrderId(), p.getId()));
        } catch (Exception e) {
            kafka.send('payment.failed', event.getOrderId(),
                new PaymentFailedEvent(event.getOrderId(), e.getMessage()));
        }
    }

    // Compensation: refund on inventory failure
    @KafkaListener(topics = 'inventory.reservation.failed')
    public void onInventoryFailed(InventoryFailedEvent event) {
        paymentService.refund(event.getOrderId());
    }
}
```

```

        kafka.send('payment.refunded', event.getOrderId(),
            new PaymentRefundedEvent(event.getOrderId()));
    }
}
// Inventory Service – Step 3: Reserve stock
@Service
public class InventorySagaStep {

    @KafkaListener(topics = 'payment.processed')
    public void onPaymentProcessed(PaymentProcessedEvent event) {
        try {
            inventoryService.reserve(event.getOrderId(), event.getItems());
            kafka.send('inventory.reserved', event.getOrderId(),
                new InventoryReservedEvent(event.getOrderId()));
        } catch (OutOfStockException e) {
            kafka.send('inventory.reservation.failed', event.getOrderId(),
                new InventoryFailedEvent(event.getOrderId(), e.getMessage()));
        }
    }
}

```

■ **Interview Tip:** Saga replaces distributed ACID transactions. It's eventually consistent — interviewers want to hear you know the trade-off.

Implement CQRS — separate Command and Query models for an Order service.

■ Scenario: CQRS — Command-Query Responsibility Segregation

```
// COMMAND side — write model (normalized, transactional)
@Service
@Transactional
public class OrderCommandService {

    public OrderId placeOrder(PlaceOrderCommand cmd) {
        Order order = Order.place(cmd); // domain logic in aggregate
        orderWriteRepo.save(order);

        // Publish event for read model synchronization
        List<DomainEvent> events = order.getDomainEvents();
        events.forEach(e -> kafka.send('order-events', order.getId(), e));
        order.clearDomainEvents();
        return new OrderId(order.getId());
    }
}

// QUERY side — read model (denormalized, optimized for UI)
@Document(collection = 'order_views') // MongoDB for flexible read model
@Data @Builder
public class OrderView {
    private String orderId;
    private String customerName; // denormalized from User service
    private String customerEmail;
    private List<OrderItemView> items; // includes product name, image
    private BigDecimal totalAmount;
    private String status;
    private String shippingAddress; // denormalized
    private LocalDateTime placedAt;
    private LocalDateTime updatedAt;
}

// Event handler updates read model from events
@Service
public class OrderViewProjection {

    @KafkaListener(topics = 'order-events')
    public void onOrderEvent(Object event) {
        if (event instanceof OrderPlacedEvent e) {
            // Build denormalized view by fetching from other services
            String customerName = userClient.getName(e.getCustomerId());
            List<OrderItemView> items = productClient.getViews(e.getProductIds());

            orderViewRepo.save(OrderView.builder()
                .orderId(e.getOrderId())
```

```

        .customerName(customerName)
        .items(items)
        .totalAmount(e.getTotal())
        .status('PENDING')
        .placedAt(e.getPlacedAt())
        .build());
    } else if (event instanceof OrderStatusChangedEvent e) {
        orderViewRepo.updateStatus(e.getOrderId(), e.getNewStatus());
    }
}
}

// QUERY service – fast reads from MongoDB read model
@Service
@RequiredArgsConstructor
public class OrderQueryService {

    public OrderView getOrder(String orderId) {
        return orderViewRepo.findById(orderId)
            .orElseThrow(() -> new OrderNotFoundException(orderId));
    }

    public Page<OrderView> getCustomerOrders(
        String customerId, Pageable pageable) {
        return orderViewRepo.findByCustomerIdOrderByPlacedAtDesc(
            customerId, pageable);
    }
}

```

■ CQRS lets you scale reads and writes independently. Write model in PostgreSQL (ACID); read model in MongoDB/Elasticsearch (flexible, fast queries).

Docker, Kubernetes & Deployment

Containerization, orchestration,
health probes & scaling

Q30 of 100

Write a production-grade Dockerfile for a Spring Boot microservice.

[illegible]

■ **Interview Tip:** Layer-based Docker builds mean only your changed code layer is rebuilt & pushed — not the 150MB dependencies. 10x faster CI/CD.

Write complete Kubernetes manifests for a Spring Boot microservice with health probes and autoscaling.

```
# deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: order-service
  namespace: production
  labels:
    app: order-service
    version: '1.0.0'
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0    # zero-downtime rollout
  selector:
    matchLabels:
      app: order-service
  template:
    metadata:
      labels:
        app: order-service
    annotations:
      prometheus.io/scrape: 'true'
      prometheus.io/path: '/actuator/prometheus'
      prometheus.io/port: '8080'
    spec:
      containers:
        - name: order-service
          image: myregistry/order-service:1.0.0
          ports:
            - containerPort: 8080
          env:
            - name: SPRING_PROFILES_ACTIVE
              value: prod
            - name: DB_URL
              valueFrom:
                secretKeyRef:
                  name: order-service-secrets
                  key: db-url
```

```

      - name: JWT_SECRET
        valueFrom:
          secretKeyRef:
            name: jwt-secrets
            key: secret
    resources:
      requests:
        memory: '256Mi'
        cpu: '250m'
      limits:
        memory: '512Mi'
        cpu: '500m'
# Liveness: is the app alive? Restart if false
livenessProbe:
  httpGet:
    path: /actuator/health/liveness
    port: 8080
  initialDelaySeconds: 60
  periodSeconds: 30
  failureThreshold: 3
# Readiness: can it serve traffic? Remove from LB if false
readinessProbe:
  httpGet:
    path: /actuator/health/readiness
    port: 8080
  initialDelaySeconds: 30
  periodSeconds: 10
  failureThreshold: 3
# Startup probe: long grace period for slow-starting apps
startupProbe:
  httpGet:
    path: /actuator/health
    port: 8080
  failureThreshold: 30
  periodSeconds: 10
# hpa.yaml – Horizontal Pod Autoscaler
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: order-service-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: order-service

```

```
minReplicas: 2
maxReplicas: 20
metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 60    # scale up when CPU > 60%
  - type: Pods
    pods:
      metric:
        name: http_requests_per_second
      target:
        type: AverageValue
        averageValue: 1000    # scale when >1000 RPS per pod
```

■ **Interview Tip:** liveness != readiness. A pod can be alive (no restart) but not ready (warming up cache). Readiness probe prevents traffic until ready.

How do you implement graceful shutdown in Spring Boot microservices for zero-downtime deployments?

```
# application.yml – graceful shutdown
server:
  shutdown: graceful          # wait for in-flight requests
spring:
  lifecycle:
    timeout-per-shutdown-phase: 30s # max 30s for each phase

# K8s sends SIGTERM -> Spring Boot finishes in-flight requests
# -> closes connections -> exits. terminationGracePeriodSeconds
# must be > timeout-per-shutdown-phase.

# In K8s deployment spec:
# spec.template.spec.terminationGracePeriodSeconds: 60
// Custom shutdown hook for additional cleanup
@Component
@Slf4j
public class GracefulShutdownHook implements DisposableBean {

    @Autowired private KafkaListenerEndpointRegistry kafkaRegistry;
    @Autowired private ThreadPoolTaskExecutor executor;

    @Override
    public void destroy() throws Exception {
        log.info('Graceful shutdown initiated...');

        // Stop consuming new Kafka messages
        kafkaRegistry.stop();
        log.info('Kafka consumers stopped');

        // Wait for async tasks to complete
        executor.setWaitForTasksToCompleteOnShutdown(true);
        executor.setAwaitTerminationSeconds(20);
        executor.shutdown();
        log.info('Executor shutdown complete');
    }
}
```

■ **Common Mistake:** Without graceful shutdown, a rolling update will kill pods mid-request causing 5xx errors to users. Always set `server.shutdown=graceful` in production.

Testing Microservices

Unit, Integration, Contract & E2E testing strategies

Q33 of 100

Explain the testing pyramid for microservices. What types of tests should you write?

The Microservices Testing Pyramid has 5 levels, from fast/cheap at the bottom to slow/expensive at the top:

Level	Type & Purpose
1. Unit	Test individual classes/methods in isolation. Mock all dependencies. Very fast. ~70% of tests.
2. Integration	Test a service with real DB/Kafka (Testcontainers). No network calls. ~20% of tests.
3. Component	Test the service as a black box via HTTP. WireMock for dependencies. ~7% of tests.
4. Contract	Verify producer/consumer API contracts (Spring Cloud Contract / Pact). ~2% of tests.
5. E2E	Full system tests across all services. Slow, flaky, expensive. Minimal. ~1% of tests.

```
// Test naming convention – describes behavior
class OrderServiceTest {

    @Test
    void should_throw_OutOfStockException_when_inventory_is_zero() { }

    @Test
    void should_send_OrderPlaced_event_when_order_created_successfully() { }

    @Test
    void should_rollback_order_when_payment_fails() { }

}
```

■ **Interview Tip:** Write tests that describe business behavior, not implementation details.
'should_send_kafka_event()' > 'testKafkaSend()'.

Write a complete integration test with Testcontainers for a service using PostgreSQL and Kafka.

```

<!-- pom.xml -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-testcontainers</artifactId>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.testcontainers</groupId>
    <artifactId>postgresql</artifactId>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.testcontainers</groupId>
    <artifactId>kafka</artifactId>
    <scope>test</scope>
</dependency>
@SpringBootTest(webEnvironment = RANDOM_PORT)
@Testcontainers
class OrderIntegrationTest {

    @Container
    static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>('postgres:16-alpine')
        .withDatabaseName('testdb')
        .withUsername('test')
        .withPassword('test');

    @Container
    static KafkaContainer kafka = new KafkaContainer(
        DockerImageName.parse('confluentinc/cp-kafka:7.6.0'));

    @DynamicPropertySource
    static void configureProperties(DynamicPropertyRegistry registry) {
        registry.add('spring.datasource.url', postgres::getJdbcUrl);
        registry.add('spring.datasource.username', postgres::getUsername);
        registry.add('spring.datasource.password', postgres::getPassword);
        registry.add('spring.kafka.bootstrap-servers', kafka::getBootstrapServers);
    }

    @Autowired private TestRestTemplate restTemplate;
    @Autowired private OrderRepository orderRepo;

    @Test
    void should_create_order_and_publish_kafka_event() throws Exception {
        // Arrange
        var request = new PlaceOrderRequest('CUST-001',

```



```

        List.of(new OrderItem('PROD-1', 2, new BigDecimal('50.00'))));

// Act
ResponseEntity<OrderDto> response = restTemplate.postForEntity(
    '/api/v1/orders', request, OrderDto.class);

// Assert HTTP response
assertThat(response.getStatusCode()).isEqualTo(HttpStatus.CREATED);
assertThat(response.getBody().getStatus()).isEqualTo('PENDING');

// Assert DB state
Optional<Order> saved = orderRepo.findById(
    response.getBody().getOrderId());
assertThat(saved).isPresent();
assertThat(saved.get().getCustomerId()).isEqualTo('CUST-001');

// Assert Kafka event published
KafkaConsumer<String, String> consumer = createTestConsumer();
consumer.subscribe(List.of('order.placed'));
ConsumerRecords<String, String> records =
    consumer.poll(Duration.ofSeconds(10));
assertThat(records.count()).isEqualTo(1);
    }
}

```

■ **Interview Tip:** Testcontainers with @Reuse (containers start once per test suite) makes integration tests fast enough to run in CI.

Implement a contract test with Spring Cloud Contract between Order Service (consumer) and Product Service (producer).

■ Scenario: Consumer-Driven Contract Testing

The consumer (Order Service) defines what it expects from the producer (Product Service). The contract is verified on both sides independently — preventing API breaking changes.

```
// contracts/product/get-product-by-id.groovy (in product-service repo)
Contract.make {
    description 'Should return product when ID exists'
    request {
        method GET()
        url '/api/v1/products/PROD-001'
    }
    response {
        status OK()
        headers { contentType applicationJson() }
        body([
            productId: 'PROD-001',
            name: 'Widget Pro',
            price: 49.99,
            stockQuantity: 100,
            category: 'ELECTRONICS'
        ])
    }
}

// Producer side – generated test (auto-generated from contract)
@SpringBootTest
@SpringBootTest
public class ProductContractTest extends ContractVerifierBase {

    @Autowired private MockMvc mockMvc;

    @BeforeEach
    void setup() {
        RestAssuredMockMvc.mockMvc(mockMvc);
    }

    // Auto-generated test verifies the contract against real controller
}

// Consumer side – stub-based test
@SpringBootTest
@AutoConfigureStubRunner(
    ids = 'com.example:product-service::stubs:8082',
    stubsMode = StubRunnerProperties.StubsMode.LOCAL
)
class OrderServiceContractTest {

    @Autowired private ProductClient productClient;
```

```
@Test
void should_fetch_product_details_from_stub() {
    ProductDto product = productClient.getProduct('PROD-001');

    assertThat(product.getProductId()).isEqualTo('PROD-001');
    assertThat(product.getPrice()).isEqualToComparingTo('49.99');
}
}
```

■ *Contract tests run in milliseconds (no network) and catch API breaking changes before they reach production. They're the safety net for microservice APIs.*

How do you test external service calls with WireMock?

```
@SpringBootTest(webEnvironment = RANDOM_PORT)
@WireMockTest(httpPort = 8090)
class OrderServiceWireMockTest {

    @Autowired private OrderService orderService;

    @Test
    void should_place_order_when_product_available() {
        // Stub product-service response
        stubFor(get(urlEqualTo('/api/v1/products/PROD-001')))
            .willReturn(aResponse()
                .withStatus(200)
                .withHeader('Content-Type', 'application/json')
                .withBody('{"productId":"PROD-001","price":49.99,"stock":10}'));

        // Stub payment-service
        stubFor(post(urlEqualTo('/api/v1/payments')))
            .willReturn(aResponse()
                .withStatus(200)
                .withBody('{"paymentId":"PAY-001","status":"SUCCESS"}'));

        // Execute
        Order order = orderService.placeOrder(
            new PlaceOrderRequest('CUST-001', 'PROD-001', 1));

        // Verify
        assertThat(order.getStatus()).isEqualTo(OrderStatus.CONFIRMED);

        // Verify interactions
        verify(exactly(1), getRequestedFor(
            urlEqualTo('/api/v1/products/PROD-001')));
    }

    @Test
    void should_activate_circuit_breaker_on_product_service_failure() {
        // Simulate 503 responses
        stubFor(get(urlMatching('/api/v1/products/.*')))
            .willReturn(aResponse().withStatus(503).withFixedDelay(5000));

        // Circuit breaker should open and return fallback
        for (int i = 0; i < 5; i++) {
            try { orderService.placeOrder(new PlaceOrderRequest()); }
            catch (ServiceUnavailableException e) { /* expected */ }
        }

        CircuitBreaker cb = circuitBreakerRegistry.circuitBreaker('product-service');
        assertThat(cb.getState()).isEqualTo(CircuitBreaker.State.OPEN);
    }
}
```

■ **Interview Tip:** WireMock tests verify: happy path, error handling, timeout handling, and circuit breaker behavior — all without a real running service.

Top 20 Quick-Fire Interview Questions

Essential answers for senior Spring Boot microservices roles

Q37 • What is the difference between `@RestController` and `@Controller`?

`@RestController` = `@Controller` + `@ResponseBody`. `@Controller` returns view names (JSP/Thymeleaf); `@RestController` returns JSON/XML data directly. Use `@RestController` for REST APIs.

Q38 • What is Spring Boot Actuator?

A module exposing operational endpoints: `/health`, `/metrics`, `/info`, `/env`, `/prometheus`. Used for liveness/readiness probes in Kubernetes and metrics scraping by Prometheus.

Q39 • What is a service mesh and how does it differ from Spring Cloud?

Service mesh (Istio/Linkerd) handles cross-cutting concerns at network level via sidecar proxy. Spring Cloud handles them in application code. Mesh is language-agnostic; Spring Cloud is JVM-only.

Q40 • Explain eventual consistency in microservices.

Data across services is temporarily inconsistent after an update but converges to a consistent state. Achieved via Saga pattern, event-driven updates, and idempotent consumers.

Q41 • What is idempotency and why is it critical in microservices?

An operation that produces the same result when called multiple times. POST `/orders` with same idempotency-key must not create duplicates. Critical for safe retries in distributed systems.

Q42 • What are the CAP theorem implications for microservices?

Distributed systems guarantee only 2 of 3: Consistency, Availability, Partition tolerance. Banks choose CP (strong consistency); social media chooses AP (high availability).

Q43 • How do you handle distributed transactions without 2PC?

Saga pattern (choreography or orchestration), optimistic locking, Outbox pattern, and eventual consistency via domain events.

Q44 • What is blue-green vs canary deployment?

Blue-green: switch ALL traffic between two environments instantly. Canary: route a % of traffic to new version (5->25->50->100%) with automatic rollback on error thresholds.

Q45 • What are `@Transactional` pitfalls to watch for?

Self-invocation bypasses proxy (no transaction). Wrong propagation level. `@Transactional` on `@Async` methods doesn't work. `REQUIRED` vs `REQUIRES_NEW` creates different transaction scopes.

Q46 • Explain Spring Cloud LoadBalancer vs Ribbon.

Ribbon (deprecated, Netflix) was replaced by Spring Cloud LoadBalancer. Supports round-robin, random, and custom strategies. Works with Eureka and Kubernetes service discovery.

Q47 • What is the Strangler Fig pattern?

Incrementally migrate a monolith to microservices by routing specific features to new services via a gateway/facade, while keeping the monolith running until fully replaced.

Q48 • How do you handle schema migration in DB-per-service?

Use Flyway or Liquibase per service. Each service owns and manages its migration scripts independently. Never share migration tooling across services.

Q49 • What is an API Gateway vs BFF (Backend-for-Frontend)?

API Gateway: single entry point for all clients. BFF: specialized gateway per client type (mobile vs web vs third-party) with tailored response formats and aggregation logic.

Q50 • How do you secure inter-service communication?

Mutual TLS (mTLS) for transport encryption + auth, JWT propagation (service accounts), network policies in Kubernetes, and API keys for internal service authentication.

Q51 • What is the difference between horizontal and vertical scaling?

Horizontal: add more instances (pods in K8s via HPA). Vertical: give existing instance more CPU/RAM. Microservices favor horizontal scaling; stateless services scale linearly.

Q52 • What is a DLQ and when do you use it?

Dead Letter Queue: messages that fail after all retries land here for analysis/replay. Use when you cannot afford message loss but also cannot block the main processing queue.

Q53 • How do you implement distributed locking across microservice pods?

Redis SETNX with TTL (Redisson library), Redis Redlock algorithm for HA, or etcd/Zookeeper. Prevents concurrent processing of the same resource across multiple pod instances.

Q54 • What is connection pooling and why does it matter?

Pool of reusable DB connections (HikariCP in Spring Boot). Without it, each request opens/closes a connection (expensive). Pool size = DB max_connections / number of pods.

Q55 • How do you implement feature flags in microservices?

Spring Cloud or Toggly libraries, cloud services (LaunchDarkly), or custom config in Config Server. Enables dark launches, A/B testing, and safe rollbacks without redeployment.

Q56 • What logging practices do you follow in microservices?

Structured JSON logging (Logback + Logstash encoder), traceId/spanId in every log line, centralized aggregation (ELK/Grafana Loki), and per-profile log levels (DEBUG dev, WARN prod).
